

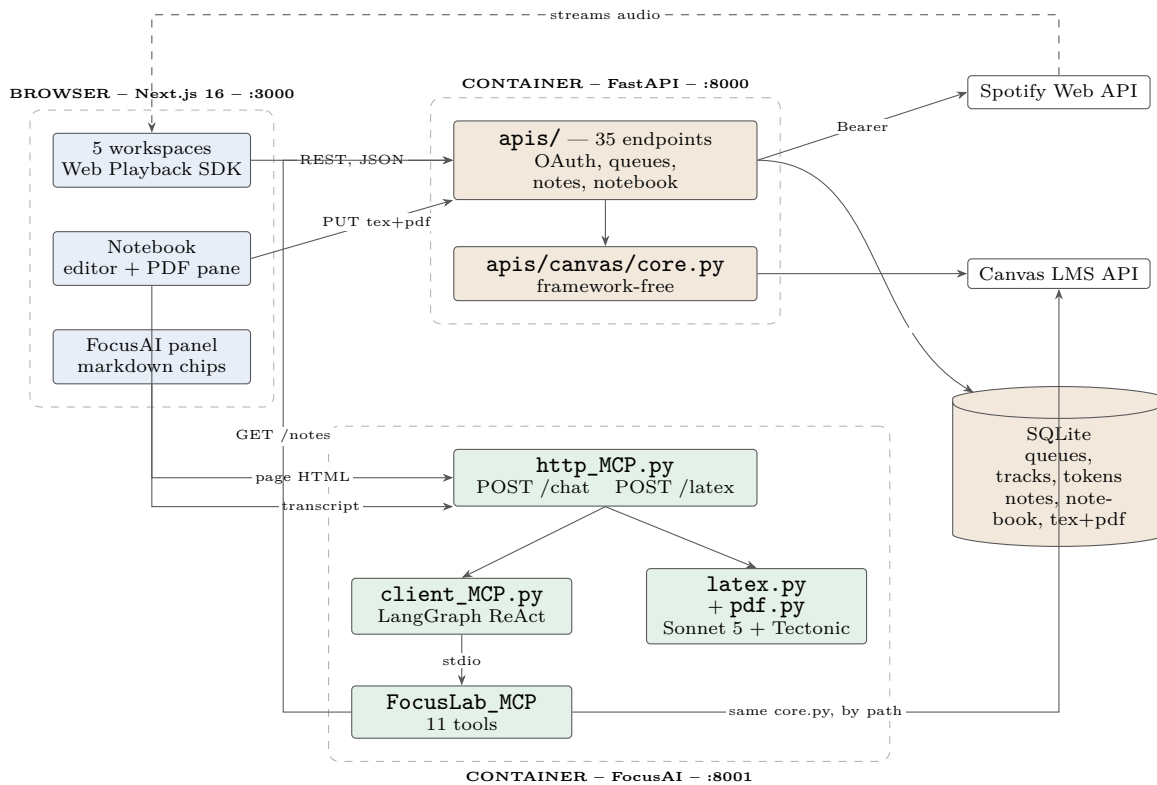
FocusLab — Architecture Defense Sheet

every branch someone can pull on, and the answer

0. Numbers

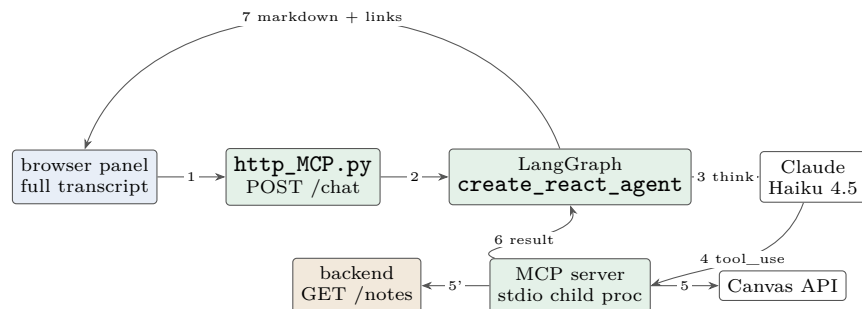
Claim	Value	Where it comes from
Services	3	<code>docker-compose.yml</code> : frontend :3000, backend :8000, agent :8001
REST endpoints	35	canvas 1 + notebook 7 + queues 8 + keys 2 + spotify 12 + todo 5
MCP tools	11	<code>FocusLab_MCP/server.py</code> , 9 Canvas + 2 notes
Lines of code	≈10,300	py + ts + tsx across backend, agent, MCP, frontend/app
Models	2	Haiku 4.5 (chat agent), Sonnet 5 (page conversion)
Containers rebuilt	agent only	Tectonic + warm cache adds ≈110 MB to that image

1. System map



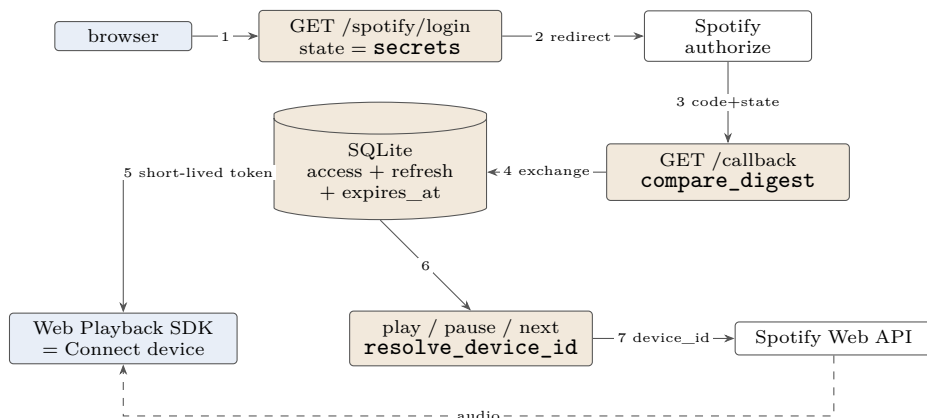
Three facts this diagram is defending: (1) the backend never touches audio — the browser is the output device; (2) one Canvas implementation, two consumers; (3) the agent reaches user data through the REST API, not the database.

2. Flow A — FocusAI chat



- **What is MCP?** Open protocol: a server declares tools with JSON schemas, a client discovers and calls them. Transport here is **stdio** — the agent spawns **server.py** as a child process.
- **Why MCP over plain tool-calling?** The tools are not welded to LangGraph. Same server works with any MCP client, and the REST API consumes the same Canvas core underneath.
- **Why LangGraph?** `create_react_agent` supplies the think→call→observe loop and message state. Model swap is one line.
- **Why Haiku 4.5 here?** Latency and cost per chat turn. Tool selection does not need frontier reasoning; the vision-heavy conversion uses Sonnet 5 instead.
- **Where is the state?** Nowhere. `/chat` is stateless: the browser owns the transcript and resends it whole. Two tabs never see each other's conversation.
- **Why is the agent built once?** `get_tools()` spawns a Python process; rebuilding per message pays that startup every turn. Cached behind an `asyncio.Lock`.
- **How do PDFs become clickable?** Canvas file endpoints return **pre-signed** URLs. The agent emits them as markdown links; the panel renders markdown. Nothing is written to disk.
- **Why not download server-side?** `download_files` writes to `~/Downloads` *inside the container* — not the user's machine. Links are the only thing that actually reaches them.
- **The 11 tools** `list_courses`, `get_modules`, `get_assignments`, `get_page_files`, `get_assignment_files`, `download_files`, `get_grades`, `get_assignment_grades`, `get_unsubmitted`, `list_notes`, `read_notes`
- **Hallucination guards** Prompt forbids inventing a course id — every id must come from a tool result; `read_notes` is illegal before `list_notes`; course matching is substring, case-insensitive, over both `name` and `title`.

3. Flow B — Spotify auth + playback

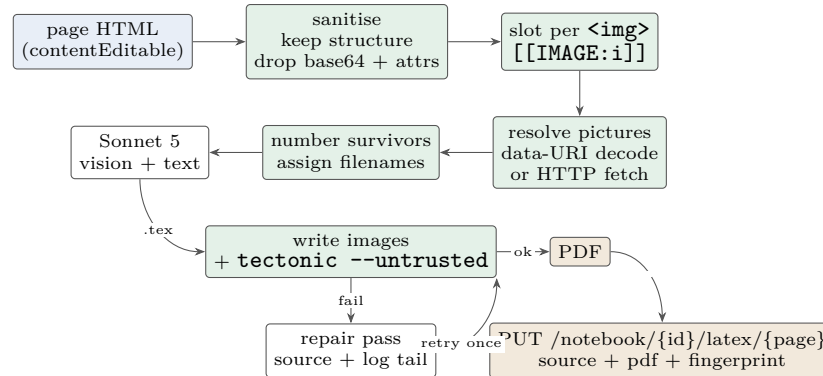


- **Which OAuth flow?** Authorization Code. Client secret lives server-side, so no PKCE requirement — the browser never sees a secret or a refresh token.
- **Why server-side state, not a cookie?** `_pending_login_states` dict: 600s TTL, capped count, **single use**, matched with `secrets.compare_digest` (constant-time). A replayed callback fails because the state was consumed.
- **What stops CSRF exactly?** An attacker cannot forge a state we handed out, and cannot reuse a real one — one state, one callback.
- **Where does refresh happen?** Server-side, before every API call: `get_valid_access_token` checks `expires_at` and

refreshes. The browser only ever gets a short-lived access token from `/spotify/token`.

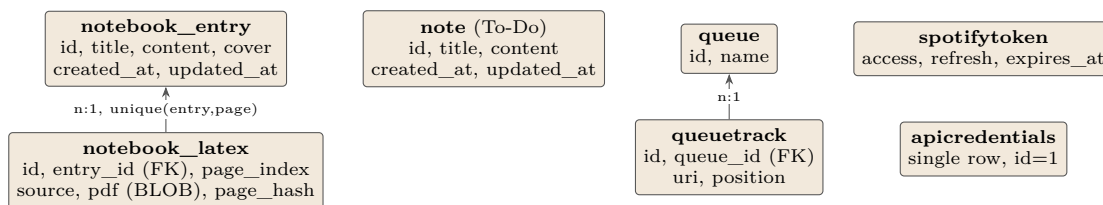
- **Why name the device explicitly?** Spotify falls back to the *active* device when a command names none. A freshly connected SDK player is *visible but not active*, so an unnamed command plays nowhere. `resolve_device_id`: prefer `is_active`, else first non-restricted, else 403 with a fix-it message.
- **Error mapping** 401 → reconnect. 403 → pass Spotify’s own wording (Premium / Extended Quota). “Restriction violated” → **409** (our polled state lagged, e.g. pausing an already-paused track). 404 → the device disappeared between resolve and command.
- **Why 127.0.0.1 not localhost?** Spotify banned `localhost` redirect URIs on 2025-11-27.
- **Known constraint** Playback control requires Premium; that is an account limit, not a bug.

4. Flow C — Notebook page → LaTeX → PDF



- **Why sanitise instead of sending raw HTML?** A pasted screenshot is megabytes of base64 — it would blow the context and the model cannot read markup anyway. Structure is *kept* (`h1--h4`, `b/i/u`, lists, inline colour) because that is exactly what maps to LaTeX commands.
- **Why slots, then numbering?** A linked picture may 404, be an SVG, or be too large. Numbers are only assigned to pictures actually in hand, so `\includegraphics` can never name a file that was not written.
- **Two kinds of image** Pasted screenshot → `data:` URI, embedded. Pasted *web page* → remote `https://` URL, fetched server-side (6 at a time, 12s timeout, type sniffed from magic bytes, not `Content-Type`).
- **Transcribe or embed?** Picture of *text* (code, table, formula) → transcribed into `lstlisting` / `tabular` / `math`. Anything else, plus logos and badges → `\includegraphics`. A code screenshot that stays a picture defeats the feature.
- **Why Tectonic over TeX Live?** One static binary + on-demand package bundle (≈110 MB total) against ≈2 GB of apt. Build-time warm compile bakes the packages in, so the first real conversion does not wait on downloads.
- **Security of compiling model output** `--untrusted`: no shell-escape, no reads outside the working directory. Each compile runs in a temp dir deleted with everything in it.
- **What if it will not compile?** One repair pass — the model gets its own source plus the log tail, told to fix only what the log names. Still failing → return the **original** transcription plus the log. The transcription is the valuable half; it is never lost to a compile error.
- **Why Sonnet 5 and not Opus?** Transcription + reading screenshots is a perception task, not a reasoning one. Verified on the hard case (code screenshot + diagram + typos + colour + escaping) — Sonnet passed all of it.
- **What does it correct?** Spelling, punctuation, grammar, and plain-text math notation (x^2 becomes x^2). Not: adding, removing, reordering, or restyling. It is a transcription contract, stated as an explicit may/may-not list in the prompt.
- **Two prompt bugs found in testing** (1) `\maketitle` prints today’s date unless `\date{}` is set — invented content. (2) Numbered `\section` on a heading that already reads “1 – Pointers” prints the number twice. Both fixed by rule, not by model choice.
- **Why store the PDF, not recompile?** A conversion costs an API call plus a compile. Re-opening a page must be free. Row keyed (`entry_id`, `page_index`), unique.
- **Staleness** FNV-1a fingerprint of the page HTML is stored with it. On open, the live page is re-fingerprinted; mismatch → the pane says the page has changed. Never auto-deletes — an out-of-date transcription still has value.
- **Caps** 24 images/request (counted *after* resolution), 60 candidates, 3.5 MB per image, 20 MB total, 120k HTML chars, 16k output tokens, 120s compile.

5. Data model



- **Why is there no pages table?** An entry's pages live inside its own `content` HTML, split on a sentinel `<hr data-page-break>`. The MCP flattener and the To-Do renderer both survive that marker; a JSON array in `content` would have rendered as garbage in both.
- **Why are Notebook and To-Do separate tables?** They were one table, so an entry appeared on both pages and an edit on either moved the other. Split into `note` and `notebook_entry`; neither model imports the other. The split *copied* rather than moved — which is what saved the data when a page was later blanked.
- **Why SQLite?** Single user, localhost, one file, zero ops. Nothing here needs concurrent writers.
- **PDF in a BLOB — really?** $\approx 12\text{--}220$ KB per page, single user. The alternative (files on disk) adds a path to keep in sync with a row. Would not survive multi-user.
- **Migration story** `create_all` builds new tables only. Column additions go through `_ADDED_COLUMNS`; the one-time note $\rightarrow$ notebook copy is guarded by “did `notebook_entry` exist before `create_all`”, which is true exactly once in a file's life.

6. Decisions someone will push on

Chose	Over	Because
Agent in its own container	inside the FastAPI backend	LLM deps (langgraph, anthropic, Tectonic) and the Anthropic key stay out of the API image; backend has no use for either
Agent reads notes over REST	direct DB access	agent is a peer of the browser, not a privileged insider — one door, one source of truth
<code>core.py</code> loaded by file path	pip-installable shared package	no packaging step, no web framework on the agent side; <code>importlib.util.spec_from_file_location</code>
<code>/latex</code> in the agent container	in the backend	that is where the key and the engine are; it stores nothing, so the containers still share no database
Frontend stores the result	agent writes to the DB	keeps <code>/latex</code> stateless and keeps writes on the same path as every other save
Tectonic	TeX Live / a hosted service	110 MB vs 2 GB; runs offline after the warm cache; no data leaves the machine
Sonnet 5	Opus / Haiku	perception task, not reasoning; verified on the hardest page in the repo
uncontrolled contentEditable	React-controlled editor	a controlled rich-text surface fights the caret on every keystroke; imperative <code>innerHTML</code> + 600 ms debounce instead
blob URL for the PDF	<code>data:</code> URL	browsers refuse to render <code>data:application/pdf</code> in a frame

7. Failure modes and limits (say these before they find them)

Failure	What happens	Why that is the right behaviour
Agent container down	panel opens, says so	UI never fails silently
Document will not compile	source shown + engine log	the transcription is the valuable half
Linked image 404s / is SVG	% [image omitted]	one dead logo must not fail a page
Image host is slow	12s timeout, skipped	a page waiting on a CDN is worse than a missing picture
Page edited after converting	amber “page has changed”	stale $\neq$ wrong; user decides whether to re-spend
Spotify device vanishes	404 mapped to a fix-it message	the state came from a poll and can lag
Two conversions at once	job counter drops the stale one	a late reply must not overwrite a newer pane
Editor not yet filled	saves are refused	see §8

8. The bug worth telling

- **Symptom.** A 37,513-character notebook entry went to 0 characters. Nobody deleted anything.
- **Mechanism.** The editor is an uncontrolled `contentEditable` written by hand. Its sync effect returned early when the entry had not loaded yet, and its dependency list (`[activeNoteId, pageIndex]`) did not change when the entry *arrived*. After a remount the div stayed empty while an entry was “open”; the next save folded that empty div back into the entry.
- **Detection.** A conversion test printed `page chars: 0`. The eslint `exhaustive-deps` warning had been pointing at this line the whole time.
- **Recovery.** The pre-split copy still existed in the To-Do table, because the split copied rather than moved. Restored via PATCH, verified 26 images intact.
- **Fix (two halves).** (1) The editor records which page it holds *on the DOM node itself* (`dataset.page`) — a freshly mounted div carries no marker, so a remount is always read as “holds nothing”, with no state to reset. (2) No save path may fold in a div that is not marked for that page.
- **Lesson to state out loud.** A lint warning that has been on screen for weeks is a bug that has not happened yet. And: a migration that copies instead of moves is a free backup.

9. Known gaps — name them first

- **No tests.** Verification is manual + scripted end-to-end runs against the live services. First thing to add: unit tests for the HTML sanitiser and the page-split helpers, which are pure functions.
- **Remote image fetch is an SSRF surface.** The agent will fetch any `http(s)` URL found in a page’s HTML, including private ranges. Single-user localhost limits the blast radius; a private-IP blocklist is the fix.
- **No auth anywhere.** Both services trust the caller. CORS is an allowlist, and an allowlist is not authentication.
- **SVG unsupported** in conversions — neither the vision API nor the typesetter reads it. `cairosvg` + `libcairo2` would close it.
- **Page indices shift.** Deleting page 2 makes the stored conversion for page 3 describe the wrong page; the fingerprint catches it as stale rather than silently mismatching.
- **Stored PDFs grow the DB** unboundedly, one row per converted page.
- **The agent only sees To-Do notes**, not Notebook entries — deliberate after the split, since the two tables number from 1 independently and ids would collide.

10. Endpoint map

Prefix	n	Routes
/spotify	12	login, callback, status, token, player, devices, search, recently-played, next, previous, pause, resume
/queues	8	list, one, create, patch, delete, play, add track, delete track
/notebook	7	list, one, create, patch, delete, GET latex/{page}, PUT latex/{page}
/notes	5	list, one, create, patch, delete
/keys	2	status, save
/canvas	1	tasks
agent :8001	2	POST /chat, POST /latex

If you can draw §1 on a whiteboard and tell the §8 story, the rest is detail you can look up.